Part C — Deployment, Cloud & Integrations

Production-Grade Edition v3 — Full DevOps, SRE & Platform Engineering Rewrite

Rebuilt around how deployment, release, and reliability are actually run at a company: containers through Kubernetes awareness, IaC and gated CI/CD, blue-green and canary releases with automatic rollback, real observability (metrics, tracing, logs, dashboards), security and dependency scanning, cost-aware platform engineering (FinOps), and event-driven async processing — layered on top of the original external-API integrations.

PHASES	TOPICS	DURATION	DIFFICULTY	PACE
7 – 13 of 14	**33 (was 24)**	**16–20 weeks**	**Intermediate → Production**	**8–12 hrs/week**

PROJECTS
3–5 per topic

What Changed in v3

- **Reorganized** the old two-phase Deployment Core / Production Ops split into five focused phases (8–12) so containers, IaC/CI-CD, observability, and security/cost each get room instead of being crammed together — plus queues promoted to its own phase (13).
- **Expanded** Docker (multi-stage, non-root, healthchecks), Compose (healthcheck-gated startup, profiles), Kubernetes (Ingress, probes, HPA), Terraform (modules, remote state, drift), GitHub Actions (matrix builds, environment protection, scan gating), Nginx (load balancing, TLS termination), Prometheus/Grafana/OpenTelemetry/logging (SLOs, dashboard-as-code, log aggregation), and Queues (RabbitMQ + SQS + Redis, idempotent consumers).
- **Turned Canary** from a v2 reasoning-only exercise into a full hands-on topic with automated rollback on SLO breach.
- **Added 12 new production topics**: Staging & Production Parity, Load Testing & Performance Engineering, Incident Response & Runbooks, Disaster Recovery/Backup & Restore, Dependency & Security Scanning, API Gateway Awareness, Feature Flags & Progressive Delivery, Cloud Cost Optimization & FinOps, Redis Queues & Streams — plus GraphQL is addressed as an optional gateway-layer pattern rather than forced in, since it was never part of ShopFlow's stack.
- Part D (Data Structures & Algorithms) renumbers to **Phase 14** — no content lost, only shifted to make room.

PHASE	FOCUS	CORE TOPICS
7	External APIs & Integrations	Payment · Email · OAuth · File Storage · AI Integration
8	Compute, Storage & Networking Foundations	Compute · Object Storage · CDN · PaaS · Domains/TLS
9	Containerization & Orchestration	Docker · Compose · Nginx/Reverse Proxy · Kubernetes
10	IaC, CI/CD & Release Engineering	Secrets · Terraform · GitHub Actions · Blue-Green/Canary · Staging Parity
11	Observability & SRE	Prometheus · Grafana · OpenTelemetry · Logging · Load Testing · Incident Response · DR
12	Security, Cost & Platform Engineering	Dependency/Security Scanning · API Gateway · Feature Flags · FinOps
13	Queues & Event-Driven Architecture	RabbitMQ · SQS · Redis Queues · Retry/DLQ · Queue Monitoring
14	Part D — Data Structures & Algorithms	Renumbered only, content unchanged

PHASE 7 OF 13

External APIs & Integrations

Third-party integration: payment, email, auth providers, and AI — the services ShopFlow depends on but does not own.

1 Payment Integration

Accepting real (or test-mode) payments and handling the asynchronous confirmation that comes back after the charge.

Subtopics Stripe/PayMongo checkout flow · payment intents · webhook confirmation · idempotency · refunds

1. Create a payment intent on the backend → confirm it client-side with the provider SDK → handle success/failure UI states → test with official test cards.
2. Build a webhook endpoint → verify the webhook signature → mark PAID only from the webhook, never the frontend → test with the provider's webhook CLI.
3. Fire the same webhook event twice → check by event ID before applying it → confirm the order is marked PAID only once → log the duplicate instead of erroring.
4. Trigger a declined test card → confirm the order stays PENDING → surface a clear error to the user → allow retry with a different card.
5. Build an admin refund action → call the provider's refund API → update order status → confirm the refund webhook is also handled.

Stripe Docs — `stripe.com/docs` | PayMongo Docs — `developers.paymongo.com/docs` | Stripe Webhooks Guide — `stripe.com/docs/webhooks`

2 Email Integration

Sending transactional emails triggered by real backend events — confirmations and receipts, not marketing email. In Phase 13 this moves off the request path entirely.

Subtopics SendGrid/Resend setup · transactional templates · triggering on backend events · deliverability basics

1. Set up SendGrid or Resend → build an HTML template with the order summary → trigger from the order-placed event → confirm it lands in a real inbox.
2. Generate a time-limited reset token → email a link containing it → build the reset page it links to → confirm the token expires and can't be reused.
3. Trigger on status change to SHIPPED → include tracking info → test via the admin status update.
4. Build one base template → reuse for confirmation and shipping notice → pass different variables per use case.
5. Simulate the provider being unreachable → confirm checkout still completes → log the failure for retry → add a retry-once mechanism.

SendGrid Docs — `docs.sendgrid.com` | Resend Docs — `resend.com/docs` | Baeldung: Email with Spring — `baeldung.com/spring-email`

3 OAuth & Social Login

Letting users sign in with Google/GitHub instead of a new password, while still issuing your own JWT.

Subtopics OAuth 2.0 authorization code flow · exchanging a code for a profile · linking to your user model · issuing your own JWT

1. Register an OAuth app with Google → redirect to Google's consent screen → exchange the code for a profile → issue your own JWT on success.
2. Detect if the OAuth email matches an existing account → link instead of duplicating → test signing in via Google after registering by email.
3. Detect a brand-new OAuth email → auto-create a user record → redirect to profile completion if needed → confirm the JWT works immediately.
4. Simulate the user clicking deny → handle the error redirect gracefully → show a clear retry path.
5. Add GitHub as a second provider → reuse the Google flow's logic → confirm both link to the same account by email.

Google OAuth 2.0 — `developers.google.com/identity/protocols/oauth2` | GitHub OAuth Apps — `docs.github.com` | Spring Security OAuth2 Client — `docs.spring.io`

4 File Storage & Uploads

Handling user-uploaded files by streaming them to cloud storage instead of the app server's own disk.

Subtopics multipart form uploads · Cloudinary/S3 SDK basics · storing the URL not the file · validating type/size

1. Build a multipart form in React → accept the file on a Spring Boot endpoint → upload to Cloudinary/S3 from the backend → store the returned URL on the Product entity.
2. Replace the file input with a drop zone → show a preview and progress → handle a non-image file gracefully.
3. Validate type on frontend and backend → cap file size → return a clear rejection error → test with an oversized file.
4. Configure an automatic thumbnail transform → store full and thumbnail URLs → use thumbnail in list view, full image in detail view.
5. Delete the product record → call the storage provider's delete API for its image → confirm no orphaned files remain.

Cloudinary Docs — `cloudinary.com/documentation` | AWS S3 Guide — `docs.aws.amazon.com/s3` | Baeldung: Multipart Upload — `baeldung.com/spring-file-upload`

5 AI API Integration

Calling an LLM API from your own backend to power a feature. Generation calls are latency-heavy and rate-limited — Phase 12's feature flags and Phase 13's queues both anchor to this topic.

Subtopics prompting from server-side code · structured/JSON responses · rate limits & cost awareness · graceful fallback

1. Send name + category to the Claude/OpenAI API → request a generated description → show it as an editable admin suggestion → handle the call failing gracefully.
2. Prompt for strict JSON output → parse it on the backend → validate the shape before saving → handle malformed JSON without crashing.
3. Accept a natural-language query → extract structured filters via the AI API → run those filters against real product search → return real results, not AI-generated ones.
4. Add a per-user rate limit → cache repeated identical requests briefly → log token usage per call → add a daily cap with a friendly fallback.
5. Simulate the API being down/timing out → confirm the rest of the app still works → show a clear unavailable state instead of an error page.

Anthropic API Docs — `docs.claude.com` | OpenAI API Docs — `platform.openai.com/docs` | Anthropic Prompt Engineering — `docs.claude.com`

FINAL PROJECT — PHASE 7 PayMongo/Stripe payment flow with webhook-driven PAID status · order-confirmation + shipping emails · Google OAuth alongside email/password · drag-and-drop product image upload on Cloudinary · AI-assisted search bar powered by the Claude API.

PHASE 8 OF 13

Cloud Compute, Storage & Networking Foundations

Where ShopFlow actually runs once it isn't localhost — compute, object storage, edge caching, hosting, and the domain/TLS layer users hit first.

1 Cloud Compute Basics

Awareness of the major compute models, hands-on with at least one, so a bill or an infra diagram makes sense later.

Subtopics AWS EC2/Elastic Beanstalk · Google Cloud Run/App Engine · Azure App Service · compute vs PaaS vs serverless

1. Launch a free-tier EC2 instance → SSH into it → install Java and run your Spring Boot JAR manually → hit the API over its public IP.
2. Package the app for Elastic Beanstalk → deploy via the EB CLI/console → confirm it runs behind a managed load balancer → compare effort to raw EC2.

3. Build your Docker image → deploy it to Cloud Run → confirm it scales to zero when idle → compare cost/complexity to EC2.
4. Note setup time, pricing model, and ease of use for each provider → decide your default for a personal vs. a company project.
5. Terminate every resource you created → confirm nothing is running in the billing dashboard → make this a standing habit (ties into Phase 12 FinOps).

AWS EC2 Guide — `docs.aws.amazon.com/ec2` | AWS Elastic Beanstalk Guide — `docs.aws.amazon.com/elasticbeanstalk` | Google Cloud Run Docs — `cloud.google.com/run/docs`

2 Cloud Storage

Storing files outside your application server, in a service built to hold them reliably at any scale.

Subtopics AWS S3 buckets & objects · Google Cloud Storage · Cloudinary (media-focused) · public vs. private access & signed URLs

1. Create a bucket via console → upload manually to confirm it works → upload programmatically from Spring Boot via the AWS SDK → retrieve it via its URL.
2. Set the bucket private by default → generate a pre-signed URL with a short expiry → confirm the file isn't accessible without it, is with it, until it expires.
3. Create a GCS bucket → upload/retrieve from your backend → compare the SDK to S3.
4. Revisit your Phase 7 upload integration → add an automatic resize/format transform → decide S3 vs. Cloudinary for product images.
5. Read the provider's pricing page → set a lifecycle rule to auto-delete after N days in a test bucket → delete all test buckets when done.

AWS S3 Guide — `docs.aws.amazon.com/s3` | Google Cloud Storage Docs — `cloud.google.com/storage/docs` | Cloudinary Docs — `cloudinary.com/documentation`

3 CDN

Serving static assets from edge locations close to the user instead of your origin — the difference between a site that feels fast everywhere and one that's only fast near your host's region.

Subtopics what a CDN caches · origin vs. edge · cache invalidation on deploy · CDN in front of S3/Cloudinary vs. built into your PaaS

1. Check response headers on your Vercel/Netlify-deployed frontend → identify the CDN provider from the headers → note which assets are cached vs. not.
2. Put CloudFront (or equivalent) in front of your S3 bucket → confirm images load faster on repeat requests → confirm the bucket itself can stay private.
3. Change a static asset → redeploy → confirm the CDN serves the new version, not a stale cached one → invalidate manually if it doesn't.

MDN: CDN Overview — `developer.mozilla.org/en-US/docs/Glossary/CDN` | AWS CloudFront Docs — `docs.aws.amazon.com/cloudfront` | Cloudflare Learning: CDN — `cloudflare.com/learning/cdn`

4 PaaS Hosting

The fastest path from GitHub repo to live URL — platforms that handle server management for you.

Subtopics frontend hosting (Vercel/Netlify) · backend hosting (Railway/Render) · auto-deploy on push · preview URLs per PR

1. Connect the repo to Vercel/Netlify → configure the build command/output dir → confirm auto-deploy on push → open a PR and confirm a preview URL.
2. Connect the backend repo/Docker image → set env vars in the dashboard → confirm the API is reachable → connect a managed PostgreSQL instance.
3. Add a build-time API base URL variable → confirm prod frontend calls prod backend → confirm local dev still points at localhost.
4. Open a PR with a visible change → confirm a unique preview URL → merge and confirm it promotes to production.
5. List what the PaaS handled automatically → list what you did manually on EC2 → decide which you'd pick for ShopFlow, and why.

Vercel Docs — `vercel.com/docs` | Netlify Docs — `docs.netlify.com` | Railway Docs — `docs.railway.com`

5 Domains, HTTPS & TLS

Pointing a real domain at ShopFlow and making sure every connection to it is encrypted end to end, with certificates that renew themselves.

Subtopics DNS records (A/CNAME/MX) · custom domains · TLS certificates & the ACME protocol · HSTS · why HTTPS is non-negotiable

1. Add the CNAME/A record your host specifies → wait for DNS propagation → confirm the domain loads the frontend.
2. Add a CNAME for api.yourdomain.com → update the frontend's API base URL → confirm CORS allows the frontend domain.
3. Confirm a valid certificate in the browser → inspect who issued it (e.g. Let's Encrypt) and how auto-renewal via ACME works.
4. Load the site over plain http:// → confirm it force-redirects → configure the redirect explicitly if it doesn't → enable HSTS and explain what it prevents.
5. Look up your A, CNAME, and MX records → explain what each does → explain why an A and CNAME can't both target the root.

Cloudflare: DNS Records — `cloudflare.com/learning/dns` | Let's Encrypt / ACME — `letsencrypt.org/how-it-works` | MDN: What Is HTTPS — `developer.mozilla.org/en-US/docs/Glossary/HTTPS`

FINAL PROJECT — PHASE 8 One compute resource and one storage bucket provisioned and torn down cleanly · product images served through a CDN-backed private bucket · frontend and backend each live on a PaaS · custom domain with an auto-renewing TLS certificate and forced HTTPS + HSTS.

PHASE 9 OF 13

Containerization & Orchestration

Packaging ShopFlow so it behaves identically everywhere, putting a real proxy in front of it, and building enough Kubernetes fluency to read a company's infra diagram.

1 Docker & Containerization

EXPANDED

Packaging the app plus everything it needs to run into one portable, production-shaped image — not just "it builds," but small, non-root, and self-checking.

Subtopics images vs. containers · multi-stage builds · image size & layer caching · non-root users · HEALTHCHECK · .dockerignore

1. Write a Dockerfile for Spring Boot → build with docker build → run with docker run and hit the API → confirm parity with local Maven run.
2. Convert it to a multi-stage build (JDK build stage → slim JRE runtime stage) → compare image size before/after → add a .dockerignore and confirm the build context shrinks.
3. Add a dedicated non-root USER to the final stage → add a HEALTHCHECK instruction → confirm docker ps shows a health status, not just 'running'.
4. Move DB credentials and JWT secret to env vars → pass via compose environment/.env → scan the built image (docker history / dive) and confirm no secret layer exists.
5. Add a named volume for Postgres data → stop/remove the container → start a new one against the same volume → confirm data survived.

Docker Get Started — `docs.docker.com/get-started` | Dockerfile Reference — `docs.docker.com/reference/dockerfile` | Docker: Multi-stage Builds — `docs.docker.com/build/building/multi-stage`

2 Docker Compose for Multi-Service Apps

EXPANDED

Running the full stack — backend, frontend, database, and now a proxy — as one reproducible local environment that mirrors production shape.

Subtopics service definitions & networking by name · healthchecks & depends_on: condition · override files per environment · Compose profiles

1. Write a compose file with backend, frontend, PostgreSQL, and Nginx → network services by name → docker compose up and confirm it all works together.
2. Add a healthcheck to the Postgres and backend services → set depends_on: condition: service_healthy on the services that need them → confirm the backend no longer boots before the DB is ready.
3. Split into docker-compose.yml (base) and docker-compose.override.yml (local-only extras like hot reload) → confirm compose merges them automatically.
4. Add a Compose profile for an optional service (e.g. a local mail-catcher) → run with and without --profile → confirm it's opt-in.
5. Diff your local compose file against your production container topology → note every deliberate difference and why it's safe.

Docker Compose Docs — `docs.docker.com/compose` | Compose Healthcheck & depends_on — `docs.docker.com/compose/compose-file/05-services` | Compose Profiles — `docs.docker.com/compose/how-tos/profiles`

3 Reverse Proxies, Nginx & Load Balancing

EXPANDED

The layer that actually receives internet traffic in front of ShopFlow — routing, TLS termination, static files, rate limiting, and (once you run more than one backend instance) load balancing between them.

Subtopics reverse proxy vs. load balancer · Nginx as a static file server · path-based routing · TLS termination at the proxy · upstream load balancing · rate limiting

1. Run Nginx as a container alongside the backend → proxy_pass /api requests to the backend service → confirm the backend is no longer directly exposed.
2. Copy the production React build into an Nginx image → configure try_files for client-side routing → confirm deep links (e.g. /product/12) don't 404 on refresh.
3. Route / to the frontend and /api to the backend from one Nginx config on one port/domain → confirm both work → explain why this avoids CORS entirely.
4. Define an upstream block with two backend container replicas → confirm Nginx round-robins between them → kill one and confirm requests still succeed.
5. Add an Nginx limit_req zone → hit an endpoint past the limit → confirm 429s are returned before the request reaches your app → terminate TLS at Nginx instead of the app.

Nginx Docs — `nginx.org/en/docs` | Nginx Reverse Proxy Guide — `docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy` | Nginx Load Balancing — `docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer`

4 Kubernetes Awareness

EXPANDED

Enough hands-on to read a job posting or a company infra diagram and know exactly what's being described — pods, config, ingress, and self-healing, not a full production K8s deployment.

Subtopics pods, Deployments, Services, Ingress · ConfigMaps & Secrets objects · readiness/liveness probes · Horizontal Pod Autoscaler · namespaces · when a team actually needs it vs. when PaaS is enough

1. Spin up a local cluster with minikube/kind → kubectl get nodes to confirm it's running.
2. Write a minimal Deployment YAML for your backend image → mount config via a ConfigMap and secrets via a Secret object, not hardcoded env → kubectl apply and confirm it's Running.
3. Add a Service to expose it → add liveness and readiness probes hitting your health endpoint → kill the process inside a pod and watch Kubernetes restart it.
4. Add an Ingress resource routing a hostname to the Service → confirm it resolves through the cluster's ingress controller.
5. kubectl scale to 3 replicas → watch them come up → optionally configure a Horizontal Pod Autoscaler on CPU → in 3-4 sentences, write down what problem Kubernetes solves that Compose doesn't, and note that most solo/small-team projects should stay on PaaS until they outgrow it.

Kubernetes Docs — kubernetes.io/docs | kubectl Quick Reference — kubernetes.io/docs/reference/kubectl | minikube — minikube.sigs.k8s.io

FINAL PROJECT — PHASE 9 ShopFlow fully containerized with multi-stage, non-root, health-checked images · Compose stack (backend + frontend + Postgres + Nginx) with healthcheck-gated startup · Nginx terminating TLS and load-balancing two backend replicas · one component running as a Kubernetes Deployment behind a Service and Ingress locally, purely for the exercise.

PHASE 10 OF 13

Infrastructure as Code, CI/CD & Release Engineering

Secrets, provisioning, pipelines, and the actual mechanics of shipping a change to production without downtime — and undoing it fast when something's wrong.

1 Secrets & Environment Management

EXPANDED

Keeping API keys, DB passwords, and JWT secrets out of source control and out of the container image itself — the single most common real-world security mistake junior developers make.

Subtopics .env files & .gitignore discipline · per-environment config (dev/staging/prod) · platform secret stores · a managed-vault mental model · secret rotation & least-privilege access

1. Create separate application-dev.yml / application-staging.yml / application-prod.yml profiles → select the profile via an env var at startup → confirm dev never touches prod credentials.
2. Move secrets out of .env into Railway/Render's or GitHub Actions' encrypted secret store → reference them as env vars at runtime/build time → confirm nothing sensitive is committed.
3. Read how HashiCorp Vault or AWS Secrets Manager centralizes and audits secret access at company scale → explain what it adds over a platform's built-in secret store.
4. Run a secret scanner (e.g. gitleaks) against your repo history → confirm it flags anything you accidentally committed before → rotate any real credential that was ever exposed.
5. Rotate the DB password or an API key → update it only in the secret store → redeploy and confirm the app picks up the new value with zero code changes.

GitHub Actions: Encrypted Secrets — docs.github.com/actions/security-guides/encrypted-secrets | gitleaks — github.com/gitleaks/gitleaks | 12-Factor App: Config — 12factor.net/config

2 Infrastructure as Code with Terraform

EXPANDED

Defining ShopFlow's cloud resources in version-controlled, reviewable code instead of clicking through a console — so environments are reproducible and staging can mirror production exactly.

Subtopics declarative vs. imperative provisioning · providers, resources, state · modules & reuse across environments · remote state & locking · plan vs. apply · drift detection

1. Install Terraform → write a provider block for AWS/GCP → define one resource (e.g. an S3 bucket) → terraform plan then apply, confirm it exists in the console.
2. Pick a bucket or instance you created by hand earlier → define it in Terraform instead → import or recreate it → destroy the manual one.
3. Inspect terraform.tfstate → explain why it must not be committed to a public repo → set up a remote state backend (S3 + DynamoDB lock, or Terraform Cloud).
4. Extract your resources into a reusable module → instantiate it twice with different variables to represent staging and production → confirm both plans differ only where expected.
5. Change a resource by hand in the console → run terraform plan → confirm it detects the drift → run terraform destroy on a test environment and confirm every resource it created is gone.

Terraform Docs — developer.hashicorp.com/terraform/docs | Terraform AWS Provider — registry.terraform.io/providers/hashicorp/aws | HashiCorp Learn: Terraform Basics — developer.hashicorp.com/terraform/tutorials

3 CI/CD with GitHub Actions

EXPANDED

Automatically testing, scanning, and building ShopFlow on every push, so broken or vulnerable code never reaches production silently.

Subtopics workflow YAML basics · running tests on push · dependency caching & matrix builds · reusable/composite workflows · environment protection rules · gating on security scans (Phase 12) · deploying on merge

1. Add .github/workflows/ci.yml → trigger on push and pull_request → run mvn test / npm test → confirm a red X on failure, green check on pass.
2. Split into backend-test and frontend-test jobs → run them in a matrix across two Java/Node versions → cache dependencies → confirm both must pass.
3. Extract the common test-and-build steps into a reusable workflow → call it from both a PR pipeline and a release pipeline.
4. Enable branch protection on main → require CI, and the dependency/security scan job from Phase 12, to pass → confirm a failing PR literally cannot merge.
5. Add a GitHub Environment named production with a required reviewer → add a deploy job on push to main only → call the hosting provider's deploy hook/CLI → confirm a merge results in a live update only after approval.

GitHub Actions Docs — `docs.github.com/en/actions` | GitHub Actions: Environments — `docs.github.com/en/actions/deployment/targeting-different-environments` | Workflow Syntax Reference — `docs.github.com/en/actions`

4 Deployment Strategies — Blue-Green & Canary

EXPANDED

Shipping a new version with zero downtime and a fast, boring way to undo it — and, once traffic and confidence justify it, releasing to a small slice of real users before everyone.

Subtopics blue-green vs. rolling vs. canary · health checks before traffic switch · one-command rollback · weighted/canary traffic splitting · automated rollback on SLO breach · backward-compatible DB migrations

1. Deploy a new version alongside the running one on a PaaS → run a health check against it before switching traffic → flip traffic to the new version → keep the old one warm for instant rollback.
2. Deploy a deliberately broken version → detect the failure via health check or monitoring (Phase 11) → roll back to the previous known-good deploy → time how long it took.
3. Add a nullable column instead of a breaking schema change → deploy code that works with both old and new schema → backfill data → only then make the column required in a later deploy.
4. Configure a canary release routing e.g. 10% of traffic to the new version (Nginx weighted upstream, or your PaaS's native canary support) → watch its error rate → promote to 100% only if it stays clean.
5. Wire the canary's error-rate metric to an automatic rollback rule → intentionally ship a broken canary → confirm it rolls itself back without a human clicking anything.

Martin Fowler: BlueGreenDeployment — `martinfowler.com/bliki/BlueGreenDeployment.html` | Martin Fowler: CanaryRelease — `martinfowler.com/bliki/CanaryRelease.html` | Google SRE Book: Release Engineering — `sre.google/sre-book`

5 Staging & Production Environment Parity

NEW

Making sure the environment you test in actually resembles the one you ship to — the gap between staging and production is where most "worked on my machine" incidents come from.

Subtopics 12-factor dev/prod parity · promoting a build artifact through environments unchanged · environment-specific config vs. environment-specific code · decoupling deploy from release with feature flags

1. Stand up a staging environment on your PaaS with its own database, distinct from production → confirm it uses the same container image build pipeline as prod.
2. Build the artifact (JAR/Docker image) exactly once in CI → promote that same artifact from staging to production instead of rebuilding → confirm nothing about the code changes between environments, only config.
3. Audit your codebase for any if (environment === 'prod') branching in application logic → replace it with configuration or a feature flag (Phase 12) instead.
4. Run your full test suite and a smoke test against staging before allowing a promotion to production → gate the production deploy job in GitHub Actions on that smoke test passing.

12-Factor App: Dev/Prod Parity — 12factor.net/dev-prod-parity | Martin Fowler: Deploy vs. Release — martinfowler.com/bliki/FeatureToggle.html | Google SRE Book: Release Engineering — sre.google/sre-book

FINAL PROJECT — PHASE 10 All secrets in a platform secret store, none in git, rotated at least once · core infra (bucket + compute + database) defined in reusable Terraform modules for both staging and prod · GitHub Actions pipeline running tests, dependency scans, and matrix builds, gated by branch protection, deploying on merge with a required reviewer · a practiced rollback with a documented time-to-recover · a real canary release with automatic rollback on an error-rate breach · a staging environment that promotes the exact same build artifact to production.

PHASE 11 OF 13

Observability & Site Reliability Engineering

Knowing what's happening inside a live system, proving it under load, and having a rehearsed plan for the day it breaks anyway.

1 Metrics & Prometheus

EXPANDED

Numbers about the system over time — request rate, latency, error rate, resource usage — collected so you can graph and alert on them, not just glimpse one in a log line.

Subtopics counters, gauges, histograms · the four golden signals · Micrometer in Spring Boot · Prometheus's scraping model · SLOs & error budgets · Alertmanager

1. Add Micrometer + the Prometheus registry → expose /actuator/prometheus → confirm request-count and latency metrics appear.
2. Run Prometheus via Docker → point it at your app's metrics endpoint → confirm it's scraping on the target's status page.
3. Write a Prometheus query for request rate, one for p95 latency, and one for error rate → explain what each tells you that logs alone don't.
4. Add a custom business metric (e.g. orders_placed_total) → trigger a few test orders → confirm the counter increments in Prometheus.
5. Define a simple SLO (e.g. 99% of requests under 300ms) → configure an Alertmanager rule that fires when the error budget is at risk, not just when something is already down.

Prometheus Docs — prometheus.io/docs | Micrometer Docs — micrometer.io/docs | Spring Boot Actuator — docs.spring.io/spring-boot/reference/actuator

2 Grafana Dashboards

EXPANDED

Turning Prometheus's raw numbers into a dashboard a human can glance at during an incident — and defining that dashboard as code so it survives a rebuild.

Subtopics connecting a data source · building panels · dashboard variables · alert rules from a panel · dashboard-as-code (JSON/provisioning)

1. Run Grafana via Docker → add Prometheus as a data source → build a panel for request rate and one for p95 latency.
2. Import a JVM/Spring Boot community dashboard → confirm it populates from your app's metrics → trim it to what's actually useful for ShopFlow.
3. Create an alert rule for error rate > X% → trigger it by generating errors → confirm the alert fires in Grafana and routes to a channel you can see.
4. Export your dashboard as JSON → commit it to the repo → provision it automatically the next time Grafana starts, instead of rebuilding it by hand.

Grafana Docs — grafana.com/docs | Grafana + Prometheus Getting Started — grafana.com/docs/grafana/latest/getting-started | Grafana Provisioning — grafana.com/docs/grafana/latest/administration/provisioning

3 OpenTelemetry & Distributed Tracing

EXPANDED

Following a single request as it crosses your frontend, backend, database, and any third-party or queued call — essential the moment your system is more than one service.

Subtopics spans & traces · context propagation across services and queues · OpenTelemetry SDK basics · exporting to a tracing backend

1. Add the OpenTelemetry Java agent or SDK → auto-instrument HTTP requests and DB calls → confirm spans are generated for a single request.
2. Export traces to Jaeger or a hosted backend → make a request that hits the DB and an external API → view the full trace as one waterfall, not scattered logs.
3. Trigger a deliberate error deep in the call chain → find its trace → confirm you can see exactly which span failed and why.
4. Propagate trace context through a queued job from Phase 13 (e.g. the order-email worker) → confirm the async work still shows up on the same trace as the request that triggered it.

OpenTelemetry Docs — `opentelemetry.io/docs` | OTel Java Instrumentation — `github.com/open-telemetry/opentelemetry-java-instrumentation` | Jaeger Tracing — `jaegertracing.io`

4 Logging & Error Tracking

EXPANDED

Finding out about production problems from a dashboard, not from an angry user — structured, aggregated, and correlated with the traces and metrics above.

Subtopics uptime monitoring · error tracking (Sentry) · structured JSON logging · log aggregation (Loki/ELK awareness) · correlating logs with a trace ID

1. Sign up for a free uptime monitor → point it at your frontend and backend URLs → configure a downtime alert → test by stopping the backend briefly.
2. Add Sentry to frontend and backend → trigger a deliberate error in each → confirm it appears with a full stack trace → add user context.
3. Switch SLF4J to structured JSON output → include the current trace ID in every log line → search/filter logs for one specific request across services.
4. Read how a log aggregator (Loki or the ELK stack) centralizes logs from many containers into one searchable place → explain why grepping individual containers doesn't scale past one service.
5. Configure an alert for an error-rate spike, not just downtime → trigger several errors in a short window → confirm the alert fires.

Sentry Docs — `docs.sentry.io` | Grafana Loki Docs — `grafana.com/docs/loki/latest` | UptimeRobot — `uptimerobot.com`

5 Load Testing & Performance Engineering

NEW

Proving ShopFlow holds up before real users find the breaking point for you — baseline, stress, and soak tests against staging, not production.

Subtopics baseline vs. stress vs. soak testing · virtual users & ramp-up · k6/Locust · reading p50/p95/p99 under load · finding the actual bottleneck

1. Write a k6 or Locust script that hits the product listing and checkout endpoints → run a baseline test at normal load → record p50/p95/p99 latency.
2. Ramp virtual users up until error rate or latency crosses your SLO → identify the actual bottleneck (DB connections, CPU, an unindexed query) using the metrics from earlier in this phase.
3. Fix the bottleneck → re-run the identical test → confirm the ceiling moved and by how much.
4. Run a soak test at moderate load for an extended period → watch for memory growth or connection leaks that only show up over time, not in a short burst.

k6 Docs — `grafana.com/docs/k6/latest` | Locust Docs — `locust.io` | Google SRE Book: Load Testing — `sre.google/sre-book`

6 Incident Response & Runbooks

NEW

What actually happens in the minutes after something breaks — a rehearsed process beats improvising in production, and a runbook beats trying to remember the fix from last time.

Subtopics severity levels · on-call basics · the incident commander role · writing a runbook · blameless postmortems

1. Define 2-3 severity levels for ShopFlow (e.g. SEV1: checkout is down, SEV2: degraded but usable) and what response each demands.
2. Write a runbook for one real failure mode you've hit in this roadmap (e.g. "payment webhook stops arriving") with concrete diagnostic steps and a fix, not just a description of the problem.
3. Simulate the incident on purpose → follow your own runbook step by step → time how long recovery took → fix any step that was unclear or wrong.
4. Pick a bug you fixed earlier in the roadmap → write a short blameless postmortem: what happened, why, what changed, and one monitoring or process improvement that would have caught it sooner.

Google SRE Book: Incident Response — `sre.google/sre-book` | PagerDuty: Incident Response Docs — `response.pagerduty.com` | Atlassian: Incident Management Handbook — `atlassian.com/incident-management`

7 Disaster Recovery, Backup & Restore

NEW

Assuming the database, the region, or the whole deployment disappears — and having a tested, timed way to bring ShopFlow back rather than a hope that backups exist.

Subtopics RTO & RPO · automated database backups · point-in-time recovery · restore drills · multi-region/failover awareness

1. Define a target RTO (how long you're down) and RPO (how much data you can lose) for ShopFlow → explain what backup frequency your RPO actually requires.
2. Configure automated daily backups for your managed PostgreSQL instance → confirm point-in-time recovery is enabled, not just full-day snapshots.
3. Deliberately drop a table or corrupt test data in a non-production database → restore from backup → time the full restore, not just the backup existing.
4. Read how multi-region failover works at a high level (active-active vs. active-passive) → explain, without building it, what it would take to fail ShopFlow over to a second region.

AWS Backup Docs — `docs.aws.amazon.com/aws-backup` | PostgreSQL: Continuous Archiving & PITR — `postgresql.org/docs/current/continuous-archiving.html` | Google SRE Book: Disaster Recovery — `sre.google/sre-book`

FINAL PROJECT — PHASE 11 Prometheus + Grafana dashboard showing the four golden signals with a dashboard-as-code definition committed to the repo · OpenTelemetry tracing across a real request including a queued job · Sentry catching real errors, uptime monitoring alerting on downtime, structured logs correlated by trace ID · a k6 load test with a documented bottleneck fixed and re-measured · a written runbook rehearsed against a simulated incident · a tested, timed database restore from backup.

PHASE 12 OF 13

Security, Cost & Platform Engineering

Catching vulnerable dependencies before they ship, controlling what sits at the edge of the API, decoupling release from deploy, and keeping the cloud bill honest.

1 Dependency & Security Scanning

NEW

Catching known-vulnerable dependencies and insecure code patterns automatically in CI, instead of finding out from a breach — gated directly into the Phase 10 pipeline.

Subtopics dependency scanning (Dependabot/Snyk) · SAST (static analysis) · container image scanning (Trivy) · OWASP Top 10 awareness

1. Enable Dependabot (or Snyk) on your repo → let it open a PR for a real vulnerable dependency → review and merge the fix.

2. Add a SAST step to your GitHub Actions workflow (e.g. CodeQL) → introduce a deliberately unsafe pattern (like string-concatenated SQL) → confirm it's flagged.
3. Scan your production Docker image with Trivy → fix at least one flagged base-image or dependency vulnerability → rescan and confirm it's clear.
4. Read the OWASP Top 10 → map at least three items to specific defenses already in ShopFlow (parameterized queries, JWT expiry, input validation) → note any gap.
5. Add the security-scan and container-scan jobs as required checks in branch protection, alongside the tests from Phase 10.

GitHub Dependabot Docs — `docs.github.com/code-security/dependabot` | Trivy Docs — `aquasecurity.github.io/trivy` | OWASP Top 10 — `owasp.org/www-project-top-ten`

2 API Gateway Awareness

NEW

The layer some companies put in front of many backend services to handle auth, rate limiting, and routing centrally — distinct from the single-app Nginx reverse proxy in Phase 9, and where a GraphQL aggregation layer would sit if ShopFlow ever split into multiple services.

Subtopics API gateway vs. reverse proxy vs. service mesh · centralized auth & rate limiting · routing to multiple backend services · where GraphQL federation fits as an alternative to REST aggregation

1. Read how Kong, AWS API Gateway, or similar centralize auth/rate limiting/routing for many services → diagram how ShopFlow's Nginx proxy differs from a full API gateway.
2. Stand up a minimal Kong (or AWS API Gateway) instance in front of your existing backend → move rate limiting from Nginx to the gateway → confirm it still returns 429s correctly.
3. Explain, in 3-4 sentences, when ShopFlow would outgrow a single reverse proxy and need a real gateway — e.g. once there's a second backend service to route between.
4. Note where a GraphQL layer would sit relative to the gateway if ShopFlow ever exposed multiple services through one schema — no build required, this is a reasoning exercise unless you choose to prototype it.

Kong Gateway Docs — `docs.konghq.com` | AWS API Gateway Docs — `docs.aws.amazon.com/apigateway` | Microsoft: API Gateway Pattern — `learn.microsoft.com/azure/architecture/patterns/gateway-aggregation`

3 Feature Flags & Progressive Delivery

NEW

Shipping code to production without releasing it to users yet — the piece that decouples deploy (Phase 10) from release, and lets a canary or a risky AI feature be turned off instantly without a redeploy.

Subtopics flag-gated rollout · percentage/user-targeted rollout · kill switches · LaunchDarkly/Unleash · flags vs. environment branching

1. Wrap one real feature (e.g. the AI search assistant from Phase 7) behind a feature flag using Unleash or a simple config-driven flag → confirm it can be toggled without a redeploy.
2. Roll the flag out to a percentage of users or a specific user segment → confirm the rest see the old behavior unchanged.
3. Deliberately break the flagged feature in production → flip the kill switch → confirm the app recovers instantly, faster than any rollback from Phase 10.
4. Pair a flag with the canary release from Phase 10: deploy the new version everywhere, but only flip the flag on for canary traffic first.

Unleash Docs — `docs.getunleash.io` | LaunchDarkly Docs — `launchdarkly.com/docs` | Martin Fowler: Feature Toggles — `martinfowler.com/articles/feature-toggles.html`

4 Cloud Cost Optimization & FinOps

NEW

Treating cloud spend as something engineered and monitored, not discovered at the end of the month — tagging, budgets, and right-sizing resources you already provisioned in Phase 8.

Subtopics resource tagging by service/environment · budgets & billing alerts · right-sizing compute · reserved/committed vs. spot pricing · the FinOps feedback loop

1. Tag every resource you've created (compute, storage, DB) with an environment and project tag → build a cost report filtered by tag.
2. Set a monthly budget in your cloud console with an alert at 50%/80%/100% → confirm you'd actually get notified before a surprise bill.

3. Review the CPU/memory utilization of a running compute resource from Phase 8 → right-size it down (or to a spot/preemptible instance for non-critical work) → confirm the app still performs.
4. Read a FinOps 101 overview → explain the inform → optimize → operate loop in your own words → identify one ShopFlow resource that's currently over-provisioned.

AWS Cost Explorer / Budgets — `docs.aws.amazon.com/cost-management` | FinOps Foundation: FinOps Framework — `finops.org/framework` | Google Cloud: Cost Optimization — `cloud.google.com/architecture/framework/cost-optimization`

FINAL PROJECT — PHASE 12 CI pipeline blocked on dependency, SAST, and container-image scan results · a minimal API gateway handling auth/rate-limiting in front of the backend · one real feature shipped dark behind a flag and toggled live without a redeploy · every cloud resource tagged, budgeted, and right-sized with a documented before/after cost.

PHASE 13 OF 13

Queues, Async Processing & Event-Driven Architecture

Moving slow or non-critical work off the request/response cycle so a user's checkout never waits on an email provider or an AI call that might be slow.

1 Message Queue Fundamentals & RabbitMQ

EXPANDED

The producer/consumer model that decouples "something happened" from "something needs to run because of it," with RabbitMQ as the hands-on broker.

Subtopics producer/consumer basics · exchanges, queues & bindings · message acknowledgment · retry with backoff · dead-letter queues

1. Run RabbitMQ via Docker → publish an OrderPlaced event to a queue on checkout → build a worker that consumes it and sends the order-confirmation email from Phase 7 → confirm checkout responds instantly, the email arrives moments later.
2. Make the email worker fail intentionally a few times → confirm it retries with backoff → confirm it lands in a dead-letter queue after max retries instead of looping forever.
3. Move the Phase 7 AI product-description call off the request path → return immediately with a "generating…" state → update the product once the worker finishes.
4. Set message acknowledgment to manual → crash the worker mid-processing → confirm the message isn't lost and gets redelivered.

RabbitMQ Docs — `rabbitmq.com/docs` | Spring AMQP Reference — `docs.spring.io/spring-amqp/reference` | RabbitMQ: Reliability Guide — `rabbitmq.com/docs/reliability`

2 AWS SQS & Managed Queueing

EXPANDED

A fully managed alternative to running and operating your own broker — useful comparison for when a team chooses "pay for it" over "run it yourself."

Subtopics standard vs. FIFO queues · visibility timeout · long polling · SQS dead-letter queues · SQS vs. self-hosted RabbitMQ

1. Create a standard SQS queue → publish the same OrderPlaced event from the RabbitMQ project → build a consumer using the AWS SDK → confirm messages are processed and deleted after success.
2. Trigger a consumer failure → observe the message reappear after the visibility timeout expires → configure a redrive policy to a dead-letter queue after N retries.
3. Switch to a FIFO queue for an operation that must preserve order (e.g. sequential order-status updates) → confirm ordering is preserved under concurrent producers.
4. Compare setup effort, cost model, and operational burden between RabbitMQ and SQS → decide which you'd pick for ShopFlow, and why.

AWS SQS Developer Guide — `docs.aws.amazon.com/sqs` | AWS SQS: Dead-Letter Queues — `docs.aws.amazon.com/sqs/latest/dg/sqs-dead-letter-queues.html` | AWS SQS: Visibility Timeout — `docs.aws.amazon.com/sqs/latest/dg/sqs-visibility-timeout.html`

3 Redis Queues & Streams

NEW

A lighter-weight queueing option many teams already have running as a cache — useful for high-throughput, lower-durability jobs where a full broker is overkill.

Subtopics Redis as a job queue (lists/BullMQ-style) vs. Redis Streams · at-least-once vs. at-most-once delivery trade-offs · when Redis is enough vs. when you need RabbitMQ/SQS

1. Run Redis via Docker → implement a simple job queue using a Redis list (LPUSH/BRPOP) or a library like BullMQ → push and process a low-priority job (e.g. a cache-warming task).
2. Rebuild the same job using a Redis Stream with a consumer group → compare its delivery guarantees and replay capability to the simple list-based queue.
3. Write down, in your own words, when you'd reach for Redis vs. RabbitMQ vs. SQS for a given job — durability needs, throughput, and whether you already run Redis for caching.

Redis Streams Intro — `redis.io/docs/latest/develop/data-types/streams` | BullMQ Docs — `docs.bullmq.io` | Redis: Lists as a Queue — `redis.io/docs/latest/develop/data-types/lists`

4 Retry, Dead-Letter Queues & Idempotent Consumers

EXPANDED

Making sure a message that's delivered twice — which will happen — doesn't charge a card twice or send a duplicate email, across whichever broker you chose above.

Subtopics idempotency keys · exactly-once vs. at-least-once processing · exponential backoff · poison-message handling

1. Add an idempotency key (e.g. the event ID) to your order-email consumer → deliver the same message twice on purpose → confirm only one email is sent.
2. Implement exponential backoff for retries instead of immediate retry → simulate a temporarily-down email provider → confirm retries space out instead of hammering it.
3. Route a message that can never succeed (a "poison message") to a dead-letter queue after max retries → build a small admin view or log alert for messages that land there.

Idempotency in Distributed Systems — `aws.amazon.com/builders-library` | Microsoft: Retry Pattern — `learn.microsoft.com/azure/architecture/patterns/retry` | Microsoft: Competing Consumers Pattern — `learn.microsoft.com/azure/architecture/patterns/competing-consumers`

5 Monitoring Queue Health

EXPANDED

A queue silently backing up is one of the most common invisible outages — surface it as a first-class metric next to the golden signals from Phase 11.

Subtopics queue depth & consumer lag as metrics · alerting on a growing backlog · autoscaling consumers on queue depth

1. Expose queue depth (or consumer lag, for streams) as a Prometheus metric → build a Grafana panel for it next to your golden-signals dashboard.
2. Generate a burst of test orders → watch queue depth spike → confirm it drains back to zero and how long that took.
3. Add a Grafana alert for queue depth exceeding a threshold for more than N minutes → explain what an on-call engineer should do when it fires, referencing your Phase 11 runbook format.

Prometheus Docs — `prometheus.io/docs` | RabbitMQ Management & Monitoring — `rabbitmq.com/docs/management` | AWS: SQS CloudWatch Metrics — `docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-available-cloudwatch-metrics.html`

FINAL PROJECT — PHASE 13 Order emails and AI description generation both running through a queue instead of blocking checkout · idempotent, retrying consumers with a working dead-letter path on at least two of RabbitMQ/SQS/Redis · queue depth visible on the Phase 11 dashboard with an alert tied to a runbook.

PART D

Data Structures & Algorithms

Becomes Phase 14 in the master roadmap — no content lost from prior versions, only renumbered to make room for the deployment expansion above.

Part C v3 — Progress Tracker

Check off each topic once concepts are read, projects are built, and the result is folded into ShopFlow. Green NEW tags mark topics added in this revision.

PHASE 7 — External APIs & Integrations

- [ ] Payment Integration
- [ ] Email Integration
- [ ] OAuth & Social Login
- [ ] File Storage & Uploads
- [ ] AI API Integration

PHASE 8 — Compute, Storage & Networking

- [ ] Cloud Compute Basics
- [ ] Cloud Storage
- [ ] CDN
- [ ] PaaS Hosting
- [ ] Domains, HTTPS & TLS

PHASE 9 — Containerization & Orchestration

- [ ] Docker & Containerization
- [ ] Docker Compose for Multi-Service Apps
- [ ] Reverse Proxies, Nginx & Load Balancing
- [ ] Kubernetes Awareness

PHASE 10 — IaC, CI/CD & Release Engineering

- [ ] Secrets & Environment Management
- [ ] Infrastructure as Code (Terraform)
- [ ] CI/CD with GitHub Actions
- [ ] Blue-Green & Canary Deployments
- [ ] Staging & Production Parity NEW

PHASE 11 — Observability & SRE

- [ ] Metrics & Prometheus
- [ ] Grafana Dashboards
- [ ] OpenTelemetry & Tracing
- [ ] Logging & Error Tracking
- [ ] Load Testing & Performance NEW
- [ ] Incident Response & Runbooks NEW
- [ ] Disaster Recovery, Backup & Restore NEW

PHASE 12 — Security, Cost & Platform Eng.

- [ ] Dependency & Security Scanning NEW
- [ ] API Gateway Awareness NEW
- [ ] Feature Flags & Progressive Delivery NEW
- [ ] Cloud Cost Optimization & FinOps NEW

PHASE 13 — Queues & Event-Driven Arch.

- [ ] Message Queues & RabbitMQ
- [ ] AWS SQS & Managed Queueing
- [ ] Redis Queues & Streams NEW
- [ ] Retry, DLQ & Idempotent Consumers
- [ ] Monitoring Queue Health

This is the version of “deployed” that matches what a production engineering team actually maintains — containerized, provisioned from code, released progressively with a rollback plan, observable end to end, scanned for vulnerabilities, cost-aware, and resilient enough to survive losing the database. Part D (Data Structures & Algorithms) becomes Phase 14.